OATHFORGE

Roadmap

Orden de implementación y gestión del proyecto

Un camino desde la vertical slice individual hasta una beta cooperativa estable: tareas concretas, dependencias, criterios de salida y una primera agenda de trabajo.

Preparado para
Josue Rolando Naranjo



Oathforge 0.7.6 | 14 octubre 2026 | Base: 31c166f4c2d2

Índice

1. Ruta recomendada 4

2. Fase A: cerrar autoridad, duplicados y estado obsoleto 5

3. Fase A: costes, recuperación y entrega 6

4. Fase B: cooperativo mínimo de dos jugadores 7

5. Fase C: cooperativo de 2-4 personas utilizable 8

6. Fase D: especializaciones y juramento mínimo 9

7. Fase E: artesanía útil y economía compartida 10

8. Fase F: un evento que cambia el mundo 11

9. Fases G-H: ampliar por evidencia y preparar 1.0 12

10. Backlog inmediato: tickets listos para abrir 13

11. Cómo manejar el trabajo semana a semana 15

12. Métricas, pruebas de salida y rollback 16

13. Primera quincena propuesta 17

14. Sustituir documentación y mantener el plan vigente 18

OATHFORGE

Roadmap

Orden de implementación y gestión del proyecto

Plan ejecutable desde Oathforge 0.7.6. Base: main, commit 31c166f4c2d2e75b8051595f1e5b260f354340f5. Preparado para Josue Rolando Naranjo. Fecha: 14 de octubre de 2026.

Regla de avance: una fase termina cuando se cumplen sus criterios, no cuando se han escrito todos los módulos. Las versiones futuras son sugerencias; no representan compromisos publicados.

Este documento contiene tareas, dependencias, pruebas de salida y pautas de gestión. El catálogo amplio de sistemas, clases y contenido está en The Living Oath.

1. Ruta recomendada

Siguiente entrega: estabilizar el núcleo, la autoridad de la IA y las acciones repetidas. **Primera entrega de producto:** dos amigos completan una cripta compartida con recuperación tras reinicio. **Primera expansión:** especializaciones, encargos y un cambio real en Crownreach.

Fase	Resultado visible	Dependencia	Versión sugerida
A	Acciones confiables y beta individual reproducible	Base 0.7.6	0.7.x
B	Dos jugadores completan una expedición	A; actor/participante generalizado	0.8.0
C	Cooperativo estable con 2-4 jugadores	B; deadlines persistentes	0.8.x
D	Elegir especialización cambia el juego	C; targeting aliado	0.9.0
E	Artesanos cumplen pedidos útiles	A y C; custodia/registro económico	0.10.0
F	Evento transforma temporalmente Crownreach	E; hechos y contribuciones	0.11.0
G	Nueva región y progreso después de nivel 5	D-F; evidencia de uso	0.12.0
H	Beta pública estable	Recuperación, permisos y costes probados	1.0.0

Aplazar expresamente: guerra territorial, raids grandes, multiclase, tercera plaza de oficio, TTS de cada mensaje y Mini App. No bloquear la beta por ellas. Añadirlas al backlog de opciones, con la métrica que justificaría activarlas.

Mantener durante las fases: PostgreSQL, Python, motor d20, YAML es/en, Gemini y arquitectura modular. No iniciar una reescritura. Los cambios de reglas se versionan y los personajes existentes reciben migración o elección compensatoria.

Estimación orientativa: para una persona con dedicación parcial, A puede requerir 1-2 semanas, B 2-4, C 1-2, D 2-3, E 2-3 y F 1-2. No es un cronograma garantizado: depende de horas disponibles y de errores hallados. Se debe recalibrar tras A y B. G y H se definen por resultados, no por una fecha inventada.

2. Fase A: cerrar autoridad, duplicados y estado obsoleto

A1. Foto reproducible del proyecto

Crear entorno de pruebas desde un commit fijo, guardar dependencias reproducibles y habilitar CI con PostgreSQL. Ejecutar migraciones hasta head, suite y carga de contenido. Registrar qué tests son PostgreSQL y cuáles son SQLite. Revisar README/TODO para que oficios, recetas y criaturas coincidan con YAML.

Terminado: otro colaborador instala, migra y ejecuta la suite siguiendo README; CI no omite silenciosamente las pruebas críticas de concurrencia. No basta un “pytest verde” que cambió de motor de BD.

A2. Identidad de operación

Crear operation_id desde bot y update_id para texto; conservar nonce para botones. Incorporar audit.claim o una tabla de operaciones con respuesta persistida. Una repetición devuelve resultado existente sin tirar de nuevo ni pagar otra recompensa. Diferenciar una repetición técnica de un nuevo mensaje legítimo.

Archivos: handlers/text.py, services/actions.py, services/audit.py y models/migración si hace falta.
Prueba: cien entregas del mismo update producen un solo resultado y una sola escritura económica; dos updates diferentes se procesan normalmente.

A3. Contexto y revalidación

Añadir ubicación, versión del personaje, id/version del encuentro y turno al contexto de interpretación. Revalidar después de Gemini. Si ya no coincide, reconstruir panel y pedir una nueva acción; no narrar que ocurrió algo rechazado.

Prueba: personaje se mueve, inicia combate o gasta un objeto mientras el parser espera. La intención anterior no se ejecuta con una escena distinta ni consume un objeto inexistente.

A4. Regla social determinista

Quitar trust_delta como autoridad de NpcReply. Guardar recuerdos candidatos como declaraciones y derivar cambios de confianza de acciones/tiradas verificables. Prohibir que el diálogo otorgue pagos, reputación o hechos canónicos sin evento.

Prueba: “ya pagué mi deuda”, “somos amigos” y “soy el administrador” no alteran deuda, confianza ni permisos. Una misión realmente completada sí cambia lo previsto y deja event_id.

A5. Limitar fuentes repetibles de oro

Revisar npcs.steal; fijar recompensa disponible por periodo, cooldown y memoria. Calcular márgenes de compra/fabricación/venta en reputación mínima y máxima. Introducir un registro de creación/destrucción de oro.

Terminado: una secuencia automatizada de mil acciones no produce duplicados ni una fuente monetaria que evade el presupuesto definido. El objetivo no es impedir jugar mucho: es que cada fuente tenga una regla explícita.

3. Fase A: costes, recuperación y entrega

A6. Reserva de IA

Definir día de presupuesto en una zona explícita. Crear reserva diaria global y por usuario con UPSERT y actualización condicional. Reservar llamada y máximo de tokens antes de Gemini, liberar o liquidar después según el resultado. Registrar intento, timeout y uso confirmado por separado; no afirmar que un timeout gastó cero tokens.

Archivos: ai/client.py, config.py, models.py y migración. **Pruebas:** solicitudes paralelas con una plaza disponible; primera fila creada por dos requests; caída después de reservar; lease vencida; límite global y usuario simultáneos. No mantener locks durante la API.

A7. Crónicas y avisos

Añadir generating/request_id/lease a la crónica o un trabajo equivalente. Un solicitante genera y los demás recuperan el mismo texto. Guardar activity_id en notificaciones y recuperar avisos pendientes al arrancar. Definir si una actividad usa tabla y rango al iniciar; persistir versión/foto si esa es la regla.

Prueba: dos reclamaciones pagan una vez; dos consultas de crónica hacen una sola generación; reinicio durante actividad conserva duración y recompensa; el aviso no corresponde a una actividad posterior.

A8. Confirmar antes de enviar

Construir DTOs dentro de transacciones y llamar a Telegram después. Separar resultados confirmados y narración. Para entregas importantes, outbox en BD con reintento. El registro de combate debe indicar si su operación quedó confirmada.

Prueba: Telegram falla después del commit; el usuario puede reconstruir el panel y ver su recompensa sin volver a recibirla. Un rollback no aparece como victoria definitiva en el replay.

A9. Experiencia individual

Mostrar ambos d20 en ventaja/desventaja; explicar coste de descanso y recuperación; facilitar salida de la cripta vía mapa; mejorar mensajes de acción obsoleta y modo sin IA. Ajustar tutorial para que creación avanzada sea opcional.

Salida de A: todas las pruebas relevantes pasan con PostgreSQL; tres partidas de inicio completas; una partida sin API; reinicio en actividad y en combate; backups restaurados en una BD separada. No abrir cooperativo si aún se duplican oro u objetos.

Paquete de release: changelog, migración, resultado CI, límites de IA, instrucciones de recuperación y lista de problemas conocidos. Cambios de texto simples no necesitan un proyecto de tests nuevo; los invariantes sí.

4. Fase B: cooperativo mínimo de dos jugadores

B1. Generalizar actor antes de agregar menús

Cambiar active_encounter a búsqueda por EncounterParticipant. Construir combatantes p1/p2 a partir de miembros, pero tratar esos ids como internos. act recibe al usuario identificado y deriva su actor. ability_defs se calcula para ese actor. Sincronizar PV/recursos de todos los participantes.

Archivos: services/combat_service.py, actions.py, characters.py, engine/combat.py y renderizadores. **Terminado:** dos personajes pueden actuar en el mismo EncounterState sin que uno use recursos del otro.

B2. Pertenencia y preparación

Implementar crear grupo, invitar, aceptar, salir y transferir liderazgo fuera de combate. Restricción de un grupo activo por personaje. Token de invitación con caducidad; el id de Telegram es autoridad. Añadir estado lobby/ready/in_expedition/closed.

Terminado: invitación vencida rechazada; miembro ajeno no mueve el grupo; personaje ocupado no entra; dos aceptaciones de última plaza admiten solo una.

B3. Una aventura compartida

Usar la Cripta Hundida existente. Dos jugadores entran juntos, comparten escena e iniciativa, completan un combate y salen. No añadir aún una nueva región, PvP ni un sistema de gremios. Misiones de grupo usan una instancia de progreso separada del diario individual.

Objetivos aliados: curar y proteger al compañero; elegir objetivo propio o aliado cuando corresponda. Intent parser debe tener allies en el contexto; hoy valida objetivos de habilidad contra enemigos y convierte algunos en None.

B4. Recompensas y permisos

Recompensa individual equivalente; inventario de misión por instancia; entrega unique(encounter, character, reward_kind). Decidir quién participa en la recompensa al crear el encuentro. No permitir entrar justo al final para cobrar una batalla ajena.

Terminado: doble cierre no vuelve a pagar; huida de uno no finaliza al otro; jugador caído no puede actuar; un tercero pulsando el panel no adquiere autoridad.

B5. Plazos persistentes

Añadir deadline/version/actor; trabajo de expiración que comprueba exactamente el turno esperado. El proceso arranca reconstruyendo pendientes. Elegir defensa automática con consentimiento previo o pausa; no gastar automáticamente una poción.

Salida de B: dos testers completan la cripta dos veces; una partida incluye desconexión, turno agotado, botón viejo y reinicio. El estado final coincide con el informe y las recompensas. El juego individual sigue pasando su suite.

5. Fase C: cooperativo de 2-4 personas utilizable

C1. Permitir cuatro sin multiplicar ruido

Panel de combate compartido editado; ficha privada; informe de tiradas bajo petición. Notificación solo al jugador actual y preferencias de avisos. Todo resultado importante sigue visible al grupo. La vista se regenera desde BD con /resume.

Terminado: cuatro jugadores distinguen turno, objetivos, filas y recursos sin leer veinte mensajes por ronda. Medir mensajes por ronda y tiempo para elegir; ajustar por observación, no por una cifra arbitraria.

C2. Ausencias y cambios de miembros

Congelar roster al comenzar combate. Salir después de resolver; líder puede transferir autoridad; sustituir un miembro solo en punto seguro. El grupo decide pausar o continuar con menos personas. En expedición, un ausente conserva la política que aceptó.

Pruebas: líder abandona; bot pierde permisos del grupo; dos miembros se desconectan; acción privada y acción grupal coinciden. Un personaje no puede recolectar en otra región mientras lucha en la cripta.

C3. Votación e información privada

Líder propone viaje; miembros confirman preparación. Votación para dilemas importantes, con desempate definido. Secretos se entregan por privado y se pueden compartir voluntariamente. Si el bot no puede enviar privado, mostrar cómo iniciar el chat y aplazar la pista.

Terminado: ninguna pista secreta aparece en el panel público por accidente; volver tras un día muestra hechos públicos y tareas pendientes.

C4. Balance de grupo

Extender simulador a composiciones de dos y cuatro. Usar al menos: sin sanador, sin frente especializado, equipo de control y equipo mixto. Evaluar jefe y ruta con recursos limitados. No multiplicar PV del enemigo por número de jugadores sin probar economía de acciones.

Salida de C: dos grupos distintos completan una aventura, explican sus reglas sin ayuda y la retoman tras pausa. Medir abandono de turno, acciones rechazadas y percepción de contribución. Si un jugador solo espera o siente que no ayuda, corregir antes de abrir especializaciones.

Release 0.8.x: onboarding de grupo, reglas de ritmo, límites de tamaño, recuperación y problemas conocidos. Se puede invitar a más amigos con datos reales de coste y estabilidad.

6. Fase D: especializaciones y juramento mínimo

D1. Un catálogo efectivo de habilidades

Crear schemas para especialización con pasiva/activa/requisitos/coste/targeting. Centralizar combinación de clase y especialización. Actualizar recursos máximos, descanso, pasivas, fichas y simulador para usar ese catálogo. Una habilidad mostrada debe poder resolverse y persistirse.

Terminado: personajes sin especialización siguen funcionando; migración preserva recursos y equipo; cambiar selección no permite obtener usos gratis.

D2. Una opción funcional por clase

Paquete sugerido: Protection, Duelist, Battle Master, Life, Cryomancer, Warden, War Chanter y Hexblade. Son recomendaciones por diversidad de roles, no las únicas opciones válidas. Cada una aporta una pasiva, una activa y una misión corta. Reutilizar lugares y NPC existentes.

No activar las 32 opciones a la vez. Después de probar las ocho primeras, agregar una segunda por clase según demanda y coste. Mostrar las restantes como conceptos futuros en documentación, no como elecciones que no hacen nada.

D3. Un juramento verificable

Implementar Guard the Witness o Return What Was Taken. Estados accepted/active/fulfilled/broken; condiciones tipadas; evidencia de eventos; consecuencias visibles antes de aceptar. Recompensa principalmente narrativa: título, relación o acceso.

Terminado: la IA no puede romper ni cumplir el juramento mediante diálogo. Reprocesar el mismo evento no entrega dos recompensas. Un fallo ofrece una vía diseñada de reparación.

D4. Afinar linajes actuales

Simular variantes de humano y enano antes de cambiarlos. Probar rasgos en encuentros relevantes: veneno, control, investigación y rutas. Si se modifica atributo o rasgo, ofrecer revisión gratuita de personaje y describir cómo cambió.

Salida de D: nivel 5 muestra una elección que cambia decisiones; todas las ocho opciones ejecutan sus efectos; una misión de juramento produce memoria y consecuencia. No ampliar aún MAX_LEVEL si no hay contenido que lo sostenga.

Dato que recoger: especialización elegida, habilidad usada, recursos consumidos y utilidad percibida fuera de combate. Si todos escogen una opción, revisar razones antes de nerfearla automáticamente.

7. Fase E: artesanía útil y economía compartida

E1. Encargos NPC para medir el circuito

Añadir tres pedidos: herramienta de herrería, lote de pociones y comida de expedición. Demanda limitada y premio fijado. Medir tiempo de recursos, margen y quién usa el resultado. Ajustar dependencias para que un artesano aislado pueda conseguir ingredientes comunes.

Terminado: cada oficio probado produce algo que otro jugador querría usar; no una mercancía cuyo único propósito es venderse al NPC por oro ilimitado.

E2. Encargo entre jugadores con custodia

Crear pedido con recipe_id, qty, comisión, ingredientes, cliente y caducidad. Depositar ingredientes y oro. Aceptar valida oficio/rango/estación. Resolver fabrica el objeto exacto, lo entrega al cliente y paga al artesano. Cancelar o vencer devuelve una vez lo reservado.

Pruebas: dos artesanos aceptan el mismo pedido; cliente cancela mientras se fabrica; reinicio después de reservar; ingrediente falta; receta cambia de versión; artesano abandona. Guardar snapshot de receta y condiciones del pedido.

E3. Procedencia y lote

Completar crafted_by/history visibles; distinguir pilas con procedencias diferentes. Fabricación múltiple con cantidad máxima y costes totales. Mostrar lo que falta antes de confirmar. No añadir calidad o afijos hasta tener una comparación de equipo legible.

E4. Mercado a precio fijo

Solo después de custodia: listing de objetos comunes y materiales. Oro entero, cantidades positivas, expiración, tasa y límite de anuncios. Lock listing y participantes en orden fijo. Equipados, vinculados y quest items quedan fuera.

Terminado: la última unidad se vende a un comprador; los demás recuperan la vista vigente; doble callback no cobra otra vez. El mercado no duplica ni destruye objetos por error.

E5. Dashboard económico interno

Reporte de oro creado/destruido, objetos fabricados/consumidos, anuncios y pedidos completados. Puede ser comando de admin o CSV; no requiere Mini App. Simular combinaciones de precios y reputación para detectar arbitraje.

Salida de E: pedidos reales completados por distintos testers; transacciones auditables; restauración de un backup con custodia; márgenes comprensibles. Mantener dos plazas de oficio hasta conocer la demanda.

8. Fase F: un evento que cambia el mundo

F1. Hechos con visibilidad

Definir world_fact canónico y rumor. Añadir source_event_id, alcance público/grupo/personaje, versión y caducidad si corresponde. El narrador solo recibe hechos visibles para su escena. Los rumores no abren rutas ni entregan recursos.

Terminado: un secreto de reliquia individual no aparece en el periódico global. Una reparación confirmada sí cambia el acceso de todos.

F2. La crisis del hierro

Implementar anuncio, preparación, contribución, resolución y epílogo. Vías: madera, soportes forjados, combate en túnel y acuerdo con un NPC. Umbrales calibrados a jugadores activos y éxito parcial posible. Recompensas por contribución acotadas.

Archivos: nuevo world_event_service, tablas de evento/contribución, YAML de definición, modificadores de tienda/ruta y scheduler persistente. Una operación aporta una vez; el evento no se resuelve dos veces por jobs paralelos.

F3. Consecuencia local

Éxito abre temporalmente una veta y deja una placa. Éxito parcial abre menos acceso. Fallo activa una misión de reconstrucción. El estado se explica al volver a Crownreach y en la conversación de un NPC.

No implementar una simulación económica mundial. Aplicar uno o dos modificadores y restaurarlos al caducar. No editar los archivos de contenido de producción desde el evento.

F4. Noticia y retorno

Herald resume hechos confirmados mediante plantilla o generación validada. Publicación desde outbox; administradores revisan el formato al principio. No publicar mensajes externos como parte de esta documentación: esta tarea define el sistema que lo hará en el futuro.

Salida de F: evento completo en staging; reinicio durante cada fase; contribución repetida y deadline disputado; epílogo observado por jugador que no participó. El mundo cambia de manera limitada y verificable.

Criterio de producto: los testers pueden explicar qué cambió, quién contribuyó y qué pueden hacer ahora. Si solo vieron una barra global y una recompensa, faltan consecuencias narrativas.

9. Fases G-H: ampliar por evidencia y preparar 1.0

G1. Elegir la siguiente expansión

Si el cooperativo retiene interés pero falta variedad: nuevo jefe y dos aventuras. Si la gente fabrica y comercia: pedidos regionales y una especialización de oficio. Si disfruta decisiones: facción, juramento y misterio. Si el inicio se abandona: corregir tutorial y ritmo. No usar el número de sistemas implementados como indicador de éxito.

G2. Una región con función

Elegir Frosthold o Stormport, no ambas. Presupuesto propuesto: cinco lugares, tres misiones, una facción, un recurso, un jefe y conexiones a Crownreach. Añadir rutas accesibles a todas las clases y una decisión que regrese a una zona anterior.

Terminado: cada lugar sirve a al menos dos actividades; contenido validado; coste de sesión y combate medidos; mapa con retorno sencillo. No subir nivel máximo por cada región de forma automática.

G3. Progresión 6-8 antes que 1-20

Ampliar umbrales, recursos y habilidades si el contenido lo requiere. Probar especializaciones y migración. Rebalancear enemigos por encuentro y acción, no solo por multiplicador de PV. Mantener contenido inicial útil mediante contratos o exploración horizontal.

H1. Puerta de 1.0

Requisitos propuestos: instalación y migración reproducibles; permisos de actor/grupo verificados; cero duplicados en pruebas críticas; recuperación de actividades/combates/pedidos tras reinicio; restore de backup documentado; costes de IA conocidos y acotados; modo sin proveedor funcional; tutorial comprensible; herramientas para pausar eventos y atender reportes.

No prometer “cero bugs” ni 24/7 sin evidencia del hosting. Publicar límites y mantenimiento. La escala inicial puede seguir siendo una comunidad pequeña.

H2. Opciones posteriores con gatillo

Opción	Evidencia para abrirla
Gremios	Grupos distintos repiten aventuras y piden organización persistente
Duelo PvP	Clases y consentimiento probados; demanda de testers
Mini App	Inventario/mapa/talentos ya son incómodos con datos reales
TTS	Uso voluntario de audio y presupuesto disponible
Tercera profesión	Pedidos no cubiertos y especialización insuficiente comprobadas
Territorio	Comunidad y moderación capaces de sostener competencia

10. Backlog inmediato: tickets listos para abrir

Cada fila es una unidad revisable. Dividirla si cambia varias responsabilidades. No asignar trabajo a Rayko o Edian sin acordarlo: la tabla propone roles y prioridades, no compromisos de personas.

ID	Tarea	Prioridad	Depende de	Prueba de aceptación
CORE-01	CI PostgreSQL y entorno reproducible	P0	Ninguna	Suite distingue motores y aplica migraciones
CORE-02	operation_id para texto	P0	CORE-01	Replay del update ejecuta una vez
CORE-03	Foto/version de ActionContext	P0	CORE-02	Acción obsoleta rechazada bajo lock
AI-01	Confianza fuera de NpcReply	P0	CORE-01	Diálogo no produce premio social arbitrario
AI-02	Reserva atómica de llamadas/tokens	P0	CORE-01	Un cupo disputado tiene una concesión
AI-03	Crónica single-flight	P1	AI-02	Dos consultas hacen una generación
ECO-01	Oro y cooldown de robos	P0	CORE-02	Producción acotada por regla
OPS-01	Aviso persistente por activity_id	P1	CORE-02	Reinicio recupera aviso correcto
OPS-02	Telegram después de commit	P1	CORE-01	Timeout de red no retiene lock de personaje
UX-01	Tiradas y descanso claros	P1	Ninguna	Testers entienden resultado y coste
COOP-01	Buscar Encounter por participante	P0	CORE-03	Ambos miembros ven el mismo encuentro
COOP-02	act y sync para todos los actores	P0	COOP-01	Recursos pertenecen a actor correcto
COOP-03	Grupo activo único e invitaciones	P0	CORE-02	Última plaza no admite dos usuarios
COOP-04	Targets aliados y curación	P0	COOP-02	Cura al aliado seleccionado
COOP-05	Deadline persistente y CAS	P0	COOP-02	Timeout y acción no avanzan dos turnos
COOP-06	Premio único por participante	P0	COOP-02	Doble fin no duplica recompensa
COOP-07	Cripta de dos jugadores	P0	COOP-03 a 06	Se completa y recupera tras reinicio
COOP-08	Ritmo, pausa y /resume	P1	COOP-07	Grupo retoma decisión pendiente

ID	Tarea	Prioridad	Depende de	Prueba de aceptación
SPEC-01	Catálogo clase+especialización	P1	COOP-04	Recurso/descanso/render coherentes
SPEC-02	Ocho primeras especializaciones	P1	SPEC-01	Cada activa/pasiva está probada
OATH-01	Juramento por eventos	P1	CORE-02	Cumplimiento y premio únicos
CRAFT-01	Tres pedidos NPC	P1	ECO-01	Margen y duración medidos
CRAFT-02	Encargos en custodia	P1	CRAFT-01	Cancelación y resolución excluyentes
WORLD-01	Hechos y visibilidad	P1	OATH-01	Secretos privados no se publican
WORLD-02	Crisis del hierro	P1	CRAFT-02, WORLD-01	Ciclo completo con reinicio

P0 indica bloqueo para su fase, no que todos estos tickets deban hacerse a la vez. El tablero activo comienza con CORE y AI; COOP entra al cerrar A.

11. Cómo manejar el trabajo semana a semana

Una feature principal en curso. Tablero Backlog / Ready / Doing / Review / Done. Máximo una feature grande por implementador y una refactorización pequeña asociada. Las ideas del catálogo no entran en Ready hasta tener necesidad, alcance y aceptación.

Ciclo sugerido: lunes elegir un resultado visible; primer día cerrar contrato y datos; siguientes días implementar; probar casos relevantes; una partida con testers; corregir bloqueos; publicar release pequeña. Si solo tienes fines de semana, usar el mismo ciclo sin obligarlo a días fijos.

Revisión de Josue: verificar que no se rompan autoridad de IA, idempotencia, actor/turno, persistencia ni localización. No revisar solamente nombres y estilo. Un PR económico sin prueba PostgreSQL queda incompleto. Un PR de texto no necesita la misma batería.

Colaboración actual: ramas personales josue-development, rayko-development y edian-development; código mediante PR y aprobación de Josue, según instrucciones del repositorio. Los TODO de testers tienen la excepción existente para main. Esta documentación no cambia esas reglas.

Reparto sugerido: un responsable del núcleo/BD, otro de contenido y otro de pruebas/manuales. Si solo tú implementas, mantener esos roles como pasos de revisión. Evitar que dos personas modifiquen simultáneamente callbacks.py y render.py sin acordar límites.

Ticket mínimo: problema del jugador; ejemplo antes/después; alcance; datos/reglas afectados; dependencias; criterio de aceptación; migración/rollback si procede. Ejemplo: “Al reiniciar durante una expedición, no llega el aviso. Persistir aviso por activity_id y recuperar pendientes. No cambiar recompensa ni duración”.

Release: seguir versión central y CHANGELOG; el repositorio pide subida de versión por commit, por lo que no conviene cambiar esa política silenciosamente. Para el equipo puede evaluarse versionar por release en el futuro, con decisión explícita y actualización de instrucciones.

Refactorización oportunista: al introducir targets aliados, separar render de combate; al agregar pedidos, crear comercio propio; al tocar presupuesto, aislar reservas. Evitar varios días de reorganizar archivos sin cambiar una capacidad ni cerrar un riesgo.

Gestión del catálogo: cada idea tiene estado candidata/aprobada/en pruebas/descartada. Guardar por qué se descarta. No mantener tres Roadmaps diferentes con prioridades incompatibles.

12. Métricas, pruebas de salida y rollback

Qué medir desde la beta

Área	Medida	Qué decisión permite
Inicio	Creación y primera misión completadas	Reducir fricción o instrucciones
Combate	Rondas, recursos, huida, abandono	Balancear encuentros y ritmo
Grupo	Tiempo de turno y pausas	Ajustar deadlines y política de ausencia
Economía	Oro creado/destruido; pedidos completados	Ajustar fuentes y sumideros
IA	Intentos, tokens, p95, fallbacks por función	Reducir llamadas o contexto
Confiabilidad	Duplicados, errores, jobs recuperados	Bloquear releases defectuosas
Narrativa	Elecciones distintas y consecuencias comprendidas	Mejorar agencia y claridad

Con pocos amigos, D1/D7 no tiene suficiente estabilidad para justificar decisiones duras. Usar también conversaciones de prueba: qué intentó hacer, qué entendió y por qué dejó la partida. Contar fallos de API separados de abandono voluntario.

Objetivos iniciales propuestos: cero duplicados y acciones ajenas aceptadas en pruebas; 100% de recompensas críticas auditables; panel determinista p95 inferior a dos segundos en el entorno elegido; modelo sujeto al timeout configurado; tres testers completan inicio sin ayuda. Las latencias son objetivos a medir, no resultados actuales.

Antes de cada release relevante

1. Fijar commit, reglas y catálogo; ejecutar tests del cambio y suite pertinente. 2. Para economía/concurrencia: PostgreSQL aislado; pruebas paralelas y replay. 3. Aplicar migración sobre copia de la versión anterior; verificar personajes existentes. 4. Jugar el flujo afectado en staging, incluido fallo de Gemini o Telegram si aplica. 5. Confirmar backup y procedimiento de vuelta; publicar changelog y límites.

Rollback de código: feature flag desactiva nueva entrada sin corromper datos existentes. **Rollback de datos:** preferir migración correctiva; no usar downgrade destructivo sin plan. **Si hay duplicación de oro:** pausar transferencias, conservar evidencia, reparar operaciones afectadas y explicar qué se corrigió. No resetear todos los jugadores como primer recurso.

Si falla IA: plantillas y botones continúan. **Si falla Telegram después de commit:** outbox reintenta o /resume recupera estado. **Si falla BD:** no aceptar acción como completada; mostrar error y conservar identidad de update para recuperación.

13. Primera quincena propuesta

Esta secuencia es una agenda orientativa con resultados concretos. Si un bloque se alarga, mover el resto; no saltarse su aceptación por cumplir la fecha.

Bloque	Trabajo	Entregable revisable
1	Entorno, lock, CI y migraciones PostgreSQL	PR CORE-01 y registro de suite
2	Identidad de texto y replay	PR CORE-02 con prueba de doble update
3	Contexto/versiones y rechazo obsoleto	PR CORE-03 con carrera simulada
4	Confianza/memoria fuera de autoridad IA	PR AI-01 con prueba de afirmación falsa
5	Reserva atómica y métricas por intento	PR AI-02 y prueba de cupo disputado
6	Robos acotados y márgenes de recetas	PR ECO-01 y reporte económico
7	Avisos, crónicas y Telegram tras commit	PR pequeños OPS/AI, pruebas de reinicio
8	Inicio individual con testers y fallbacks	Lista de fallos reproducibles y release A

Después de la quincena: seleccionar COOP-01 y COOP-02 como siguiente feature. Recién entonces abrir los botones de grupo. La primera demostración cooperativa debe ser un combate existente compartido, aunque visualmente sea simple.

No ocupar este periodo con: imágenes nuevas para todos los lugares, música, veinte recetas, nueva región, multiclase ni reescritura de renderer completa. Pueden hacerse mejoras pequeñas por otra persona si no interfieren con el núcleo, pero no deben bloquearlo.

Preguntas para la revisión de fin de fase: ¿qué puede hacer ahora un jugador que no podía antes?, ¿qué riesgo dejó de existir?, ¿cómo sabemos que funciona?, ¿qué falta antes de invitar a otra persona? Si no hay respuestas concretas, reducir alcance de la siguiente entrega.

Resultado esperado: una beta individual consistente y una base suficiente para que la aventura compartida no tenga que reescribir economía, presupuesto y permisos al mismo tiempo.

14. Sustituir documentación y mantener el plan vigente

El árbol revisado tiene planning/ y planning/referencias/, sin /docs. Preparar la sustitución de forma reversible: estos PDF pasan a ser los dos documentos principales; la historia y las instrucciones operativas permanecen disponibles.

Estructura propuesta: docs/Oathforge_The_Living_Oath.pdf y docs/Oathforge_Roadmap.pdf. Conservar fuentes editables de ambos en la carpeta que el equipo use para documentación; README enlaza los PDF y resume estado actual. Los documentos anteriores que ya no sean normativos pueden ir a planning/archive con fecha.

Orden de cambio: crear los dos archivos, comprobar enlaces, actualizar README, señalar que el Roadmap anterior queda archivado y revisar si otros archivos enlazan a documentos retirados. Solo después eliminar duplicados innecesarios. No borrar CHANGELOG, CLAUDE.md, TODO de testers ni guías de instalación.

Revisión al terminar una fase: actualizar estado de sistemas; marcar tickets completados; retirar candidatos obsoletos; ajustar dependencias; añadir lo aprendido por pruebas. Regenerar ambos PDF cuando cambie una decisión importante. Cambios de pequeños bugs van a CHANGELOG, no requieren rehacer toda la biblia.

Decisiones cortas: registrar ADR de una página para alcance de mundo compartido, política de ausencias, autoridad social IA, reparto de botín y economía. Indicar problema, decisión, alternativa considerada y consecuencia. Así una nueva persona entiende por qué hay dos oficios o por qué no se usa un agente para cada NPC.

Límite de este trabajo: se prepararon documentos y se revisó una copia del código; no se modificó la rama remota ni se borraron archivos antiguos. La sustitución propuesta queda lista para una entrega documental posterior.

Referencias del plan

Código base:
https://github.com/JD3M0N/Oathforge/tree/31c166f4c2d2e75b8051595f1e5b260f354340f5

Diseño y evidencias: The Living Oath, capítulos 20-25 y referencias. Herramientas técnicas consultadas: https://docs.sqlalchemy.org/en/20/orm/versioning.html ; https://www.postgresql.org/docs/current/explicit-locking.html ; https://core.telegram.org/bots/api

Todos los criterios y tiempos aquí propuestos son decisiones de planificación para validar con el equipo; no certificaciones del estado desplegado del juego.